Control PID: Teoría e Implementación

Planta: Sistema *Ball and Beam* (Bola y Viga)
Plataforma: Microcontrolador STM32F4 (Bare-Metal en C)

1. Conceptos Fundamentales del Lazo PID

Un controlador PID (Proporcional, Integral, Derivativo) es un mecanismo de retroalimentación que calcula continuamente un valor de error y aplica una corrección para estabilizar un sistema. En una maqueta de *ball and beam*, el objetivo (*Set Point*) es mantener la bola en una posición específica, y el actuador es un servomotor que inclina la viga.

- **Proporcional (P) — "El Presente":** Depende del error actual (la distancia entre donde está la bola y donde quieres que esté). Si la bola está muy lejos, el servo inclina la viga bruscamente. *Problema:* Por sí solo, el motor empujará la bola, pero por inercia esta se pasará de largo, creando oscilaciones.
- **Derivativo (D) — "El Futuro":** Observa la velocidad a la que se mueve la bola. Si la bola se acerca rápidamente al centro, la acción derivativa actúa como un "freno", reduciendo la inclinación de la viga antes de que la bola llegue al objetivo para evitar el sobreimpulso.
- **Integral (I) — "El Pasado":** Suma los errores anteriores a lo largo del tiempo. Si la bola se queda estática cerca del centro (por fricción o desbalance), la acción P no será suficiente para moverla. La acción I acumulará ese pequeño error hasta generar la fuerza necesaria para llevar la bola exactamente al centro.

2. Tiempos de Integración y Derivación

Para comprender estos parámetros, partimos de la **forma estándar (o ideal)** de la ecuación PID, donde representan constantes de tiempo físicas:

$$Salida = K_p \left[e(t) + (1 / T_i) \int e(t)\, dt + T_d \cdot de(t)/dt \right]$$

- **Tiempo de Integración (T_i):** Es el tiempo exacto que tarda la acción integral en igualar la magnitud de la acción proporcional, asumiendo un error constante. Un T_i más pequeño implica una corrección más rápida y agresiva. Su equivalencia algorítmica es $K_i = K_p / T_i$.
- **Tiempo de Derivación (T_d):** Representa cuánto tiempo hacia el futuro proyecta el controlador el error basándose en su velocidad de cambio actual. Un T_d mayor aumenta el efecto de frenado. Su equivalencia algorítmica es $K_d = K_p \cdot T_d$.

Sintonización: Método Ziegler-Nichols (Lazo Cerrado)

1. **Aislar el Proporcional:** Configurar $T_i = \infty$ ($K_i = 0$) y $T_d = 0$.
2. **Buscar la Oscilación Crítica:** Incrementar gradualmente K_p hasta que la bola oscile de forma sostenida y constante. Este valor es la Ganancia Crítica (K_u).
3. **Medir el Período:** Cronometrar el tiempo de una oscilación completa. Este es el Período Crítico (T_u).
4. **Calcular los parámetros:** $T_i = 0.5 \times T_u$, $T_d = 0.125 \times T_u$, y $K_p = 0.6 \times K_u$.

3. Determinación del Tiempo de Muestreo (T_s)

La elección del tiempo de muestreo (T_s o *dt*) requiere equilibrar la teoría de control con las limitaciones físicas del hardware del microcontrolador.

Criterio Teórico (Dinámica de la Planta)

T_s debe ser entre **10 y 30 veces más rápido** que la dinámica dominante del sistema. Usando el Período Crítico (T_u) de Ziegler-Nichols, una buena aproximación es:

$$T_s \approx T_u / 10 \quad \text{a} \quad T_u / 20$$

Criterio Práctico (Limitaciones de Hardware)

- **Tiempo de lectura del sensor:** Si se usa un sensor ultrasónico, la velocidad del sonido impone un límite. Un T_s muy pequeño provocará que el lazo se ejecute antes de tener un nuevo dato.
- **Tiempo de respuesta del actuador:** Un servo PWM estándar opera a 50 Hz (20 ms). Un lazo ejecutado cada 5 ms es un desperdicio de CPU, ya que el servo ignorará los comandos intermedios.
- **Ruido de cuantización:** Con un T_s muy corto, el cambio en la lectura del sensor será casi nulo. Al dividir este pequeño salto por un T_s diminuto, la acción derivativa produce picos irreales que causan temblores violentos (*chattering*).

Conclusión: Para una maqueta *ball and beam* típica, un T_s entre **20 ms y 50 ms** (20 a 50 Hz) es ideal.

4. Implementación en STM32F4 (Bare-Metal en C)

Utilizamos una estructura para encapsular el estado y evitar variables globales, empleando aritmética de punto flotante nativa (FPU) y ejecución no bloqueante mediante `HAL_GetTick()`.

pid.c - Lógica del Controlador

```
 #include <stdint.h>

/* Estructura para encapsular el estado y los parámetros del lazo */
typedef struct {
    float kp;
    float ki;
    float kd;
    float ts;              /* Tiempo de muestreo en segundos */
    float error_previo;
    float integral;
    float limite_out_max;  /* Límites físicos del actuador (ej. grados del
servo) */
    float limite_out_min;
} PID_Controller;

void PID_Init(PID_Controller *pid, float kp, float ki, float kd,
              float ts, float max, float min) {
    pid->kp = kp;
    pid->ki = ki;
    pid->kd = kd;
    pid->ts = ts;
    pid->error_previo = 0.0f;
    pid->integral = 0.0f;
    pid->limite_out_max = max;
    pid->limite_out_min = min;
}

float PID_Compute(PID_Controller *pid, float setpoint, float medida_actual) {
    float error = setpoint - medida_actual;

    /* 1. Acción Proporcional */
    float p_term = pid->kp * error;

    /* 2. Acción Integral (Aproximación rectangular) */
    pid->integral += error * pid->ts;
    float i_term = pid->ki * pid->integral;

    /* Anti-Windup: Saturación dinámica del acumulador integral */
    if (i_term > pid->limite_out_max) {
        i_term = pid->limite_out_max;
        pid->integral = i_term / pid->ki;
    } else if (i_term < pid->limite_out_min) {
        i_term = pid->limite_out_min;
        pid->integral = i_term / pid->ki;
    }

    /* 3. Acción Derivativa */
    float d_term = pid->kd * ((error - pid->error_previo) / pid->ts);
    pid->error_previo = error;

    /* 4. Cálculo de la señal de control total */
    float salida = p_term + i_term + d_term;
```

main.c - Bucle Principal No Bloqueante

```
 #include "stm32f4xx_hal.h"

/* Capa de abstracción de hardware (Sensores y Actuadores) */
float LeerSensorDistancia(void);
void ActualizarServo(float angulo);

int main(void) {
    HAL_Init();
    /* (Inicialización de relojes, GPIO y TIMx para PWM omitida) */

    PID_Controller pid_bola;
    /* Kp=1.5, Ki=0.2, Kd=0.05, Ts=0.02s (20ms), Limites: +/- 15 grados */
    PID_Init(&pid_bola, 1.5f, 0.2f, 0.05f, 0.02f, 15.0f, -15.0f);

    const uint32_t TS_MS = 20; /* 20 ms coincide con el Ts del PID */
    uint32_t tiempo_anterior = HAL_GetTick();

    float setpoint_centro = 0.0f; /* Centro de la viga */

    while (1) {
        uint32_t tiempo_actual = HAL_GetTick();

        /* Ejecución determinista no bloqueante */
        if ((tiempo_actual - tiempo_anterior) >= TS_MS) {
            tiempo_anterior = tiempo_actual;

            float posicion_actual = LeerSensorDistancia();
            float salida_angulo = PID_Compute(&pid_bola, setpoint_centro,
posicion_actual);
            ActualizarServo(salida_angulo);
        }

        /* Espacio libre para rutinas de fondo (telemetría, displays) */
    }
}
```

Notas Finales de Implementación

- **Manejo del Anti-Windup:** Evita que el término integral crezca indefinidamente cuando el actuador ha llegado a su límite físico. Sin esto, el sistema tardaría demasiado en reaccionar cuando la bola cruce el centro.
- **Abstracción de Hardware:** Separar la lectura del sensor y la escritura del actuador en funciones independientes permite cambiar el hardware físico sin alterar el algoritmo matemático del PID.